

ArrayList FRQ Key

There were two common solutions given to this problem. The first creates a new list to hold the “good” items before averaging.

```
public double averageWithinRange(double lower, double upper)
{
    double sum = 0.0;
    int count = 0;
    ArrayList<ItemInfo> goodItems = new ArrayList<ItemInfo>();

    for (int i = 0; i < inventory.size(); i++)
    {
        if (inventory.get(i).isAvailable() && inventory.get(i).getCost() >= lower
            && inventory.get(i).getCost() <= upper)
        {
            goodItems.add(inventory.get(i));
        }
    }

    for (int i = 0; i < goodItems.size(); i++) {
        sum += goodItems.get(i).getCost();
        count++;
    }

    return sum / count;
}
```

The second uses the original list and just updates the sum and count as needed.

```
public double averageWithinRange(double lower, double upper)
{
    double sum = 0.0;
    int count = 0;
    for (int i = 0; i < inventory.size(); i++)
    {
        if (inventory.get(i).isAvailable() && inventory.get(i).getCost() >= lower
            && inventory.get(i).getCost() <= upper)
        {
            sum += inventory.get(i).getCost();
            count++;
        }
    }
    return sum / count;
}
```

A third solution, which was rare, is the best, though. Why? Because it is shorter, easier to write and much less error prone! It uses enhanced for loops.

```
public double averageWithinRange(double lower, double upper)
{
    double sum = 0.0;
    int count = 0;
    for (ItemInfo item : inventory)
    {
        double cost = item.getCost();
        if (item.isAvailable() && cost >= lower && cost <= upper)
        {
            sum += cost;
            count++;
        }
    }
    return sum / count;
}
```

Scoring your solution

What to look for

- Traverses every element of `inventory`.
- Tests availability and both bounds, inclusive (`>=` and `<=`).
- Accumulates a sum and a count of only the qualifying items.
- Returns sum divided by count, as a `double`.

Common mistakes

- Using `>` or `<`, which excludes the endpoints.
- Dividing by `inventory.size()` instead of the count of qualifying items.
- Declaring `sum` as an `int`, which loses the cents.
- Forgetting the `isAvailable()` check (the watch is the test case for this).